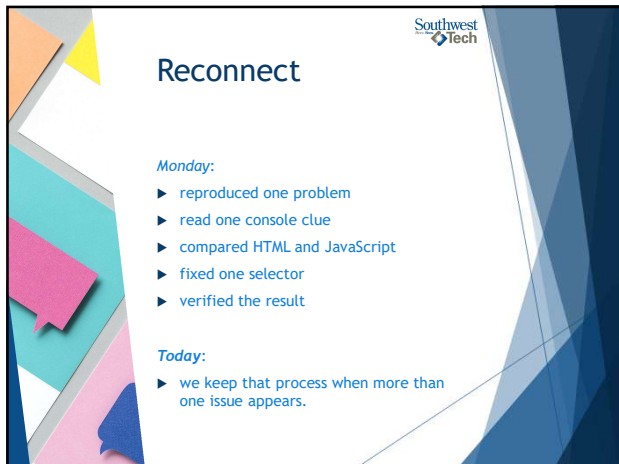




1



2



3

Southwest Tech

The Working Problem: More Than One Thing Can Be Wrong

A page can have multiple issues:

- JavaScript selector mismatch
- condition logic error
- CSS selector mismatch
- confusing user feedback

Fixing one issue may reveal the next.

That is normal.

4

Southwest Tech

Multiple Layered Issues on One Page

More than one thing can be the problem.

Status Dashboard

Click the button to load the status.

Load Status

Status Message
(nothing shows)

Looks
Button style not applied.

Layered Issues

- JS**
JavaScript selector mismatch
The script cannot find the element. Nothing happens when the button is clicked.
- Condition**
Condition logic error
After the selector is fixed, the logic prevents the message from showing.
- CSS**
CSS selector mismatch
After the message appears, the style does not apply because the CSS selector is wrong.

Multiple Issue Stack

Fixing one issue may reveal the next.

- Work step by step.
- Verify after each fix.
- Be ready for what's next.

5

Southwest Tech

What Counts as Success Today

- you choose one likely cause first
- you make one focused change
- you retest before moving on
- you document what happened
- at least two fixes are verified

6



Prioritize The First Cause

Start with the issue that blocks testing.

Good first checks:

- ▶ script file linked?
- ▶ console error present?
- ▶ selector matches HTML?
- ▶ event listener connected?

Cosmetic issues can wait until behavior works.



© 2016 Southwest Tech. All rights reserved. A licensed under CC BY-SA.

7



Debugging Priority

Check in order. One step at a time.

- **script linked?**

Is the JavaScript file connected to the page?
- **console error?**

Does the console show an error that stops code?
- **selector matches?**

Does the selector match the HTML element?
- **event connected?**

Is the event listener attached correctly?
- **condition logic?**

Is the logic allowing the code to run as expected?
- **styling issue?**

Does the styling affect what you see?



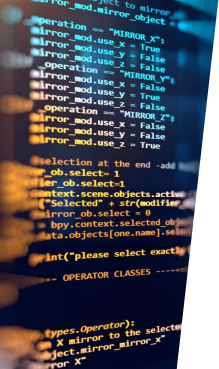
Start with what blocks testing.

-  Fix blocking issues first.
-  Verify after each change.
-  Move down the list.



Priority Ladder

8



```
...
    mirror: mirror_obj,
    mirror: mirror_obj,
    operation = "MIRROR_X",
    mirror_mod.use.x = true
    mirror_mod.use.y = false
    operation = "MIRROR_Y"
    mirror_mod.use.x = false
    mirror_mod.use.y = true
    operation = "MIRROR_Z"
    mirror_mod.use.x = false
    mirror_mod.use.y = false
    mirror_mod.use.z = true

    selection at the end - add
    obj.select-1
    var obj.select-1
    context.scene.objects.active
    "selected" : selected
    mirror_obj.select = 0
    bpy.context.scene.objects
    data.objects[one.name].select
    int("please select exactly")

-- OPERATOR CLASSES -----

types.Operator:
    X mirror to the selected
    select_mirror_mirror_X
    error X

@context:
    context.active_object is not
```



What We Will Use Today

- ▶ issue list
- ▶ priority
- ▶ console error
- ▶ `'console.log()'`
- ▶ selector check
- ▶ condition check
- ▶ CSS mismatch check
- ▶ verification note

9



Today's Multi-Issue Debugging Toolbox

Use the right tools in the right order.



Use the tools. Follow the priority. Verify the fix.

Same process. Different tools.

Identify → Prioritize → Investigate → Fix → Verify → Explain

Debugging Toolbox

10



Do Not Fix Everything At Once

Avoid This

- ▶ change selector
- ▶ change condition
- ▶ change CSS
- ▶ refresh once
- ▶ hope it works

Use This

- ▶ change one likely cause
- ▶ retest
- ▶ record result
- ▶ then choose the next issue

11



Debug Smarter, Not Harder

Two ways to handle multiple issues.

Chaotic Approach

Change Everything



Hard to know what fixed it. New problems can appear.

Calm Approach

One Fix at a Time

1. change one cause (find smallest most likely cause and change only that)
2. retest (run the same test or action)
3. record result (what changed? what stayed the same?)
4. choose next issue (use your results to decide what to tackle next)

Clear results. Steady progress. Fewer new problems.

Retesting tells you what worked. Small, targeted changes. Clear evidence from each step. Confident then, one at a time.

One Fix At A Time

12



Console Logs As Evidence

`console.log()` can answer:

- ▶ did this function run?
- ▶ what value did the input contain?
- ▶ which branch did the condition choose?
- ▶ did the code reach this line?

Use logs to test a question.
Then remove or clean them later.

13



console.log as Evidence Checkpoints

Place logs at key points to prove what your code is doing.

Simple Code Path

```

</>
User clicks button
↓
handleClick()
function starts
↓
f()
↓
Read input value
from the text box
↓
Check condition
choose a branch
↓
Show result
update the message

```

These logs are checkpoints, not final answers.
They help you see what is happening.

Console Output (Simplified)

- 1 function started
did the function run?
- 2 input value read
What value did we get?
- 3 condition branch chosen
Which branch is taken?

Each message answers the question at that step.

Use logs to test a question. ? Did it run? ? What value did we get? ? Which branch did we take? ? Record what you learn.

Console Logs As Evidence

14



Proper `console.log()` Format

Use logs that explain what you are checking.

Less useful:

- ▶ `console.log(taskText);`

More useful:

- ▶ `console.log("Task text:", taskText);`

Best habit:

- ▶ - label the value
- ▶ - log one question at a time
- ▶ - remove extra logs after debugging (or comment them out)

15

Southwest Tech

Proper Beginner console.log Formatting

Make your logs easy to read and easy to understand.

Less Useful

```
console.log(taskText);
```

What am I looking at?
What question does this answer?

Output (example)
Read the task

VS

More Useful

```
console.log("Task text:", taskText);
```

Clear label. Clear purpose.
Now I know what this value means.

Output (example)
Task text: Read the task

Three Good Logging Habits

1. Label the value
Add a short label so future you knows what it is.
2. Log one question at a time
Keep each log focused on one thing.
3. Remove extra logs later
Clean up logs once the issue is solved.

A useful log explains what you are checking. Logs help you see the program's behavior and guide your next step.

Proper Console Log Format

16

Demo

"Multi-Issue Debugging"

17

Southwest Tech

What to Watch For

- ▶ first visible symptom
- ▶ first console clue
- ▶ selector mismatch
- ▶ retest after selector fix
- ▶ condition error
- ▶ CSS issue handled after behavior works

18

Multi-Issue Debugging (Recorded Demo)
One page. Three issues. One methodical path.

Task Page

Score Checker
Enter a score and click Check.
Result: 45 (nothing shown)
Check Score

Three Issues on This Page

- 1. selector information: Select each find the button or result area.
- 2. condition error: This page prevents the result from changing.
- 3. CSS mismatch: The result text appears but has no style.

Methodical Path (One Issue at a Time)

1. fix selector: Cover the selector so elements can be found.
2. retest: Run the same action. Confirm new behavior.
3. fix condition: Cover the light so the result can show.
4. retest: Run the same action. Confirm new behavior.
5. check styling: Fix the CSS selector or style so it looks right.

What You'll See in the Demo

- Small, focused fixes: Solve one issue at a time.
- Retest after each fix: See the effect of each change.
- Clear progress: From no result to correct result and good styling.
- Some progress, Three Issues: Fix the blocker first, then the rest.

Demo: Multi-Issue Debugging

19

Retest After Each Fix

After each change, ask:

- ▶ Did the original symptom change?
- ▶ Did the console error change?
- ▶ Did a new issue appear?
- ▶ Is the current fix verified?
- ▶ What is the next smallest check?

Retesting turns edits into evidence.

20

Retesting After Each Fix
Small change. Retest. Collect evidence.

Fix 1
Make one targeted change. → Retest (Run the same action or test again). → Evidence (Record what happened and what changed).

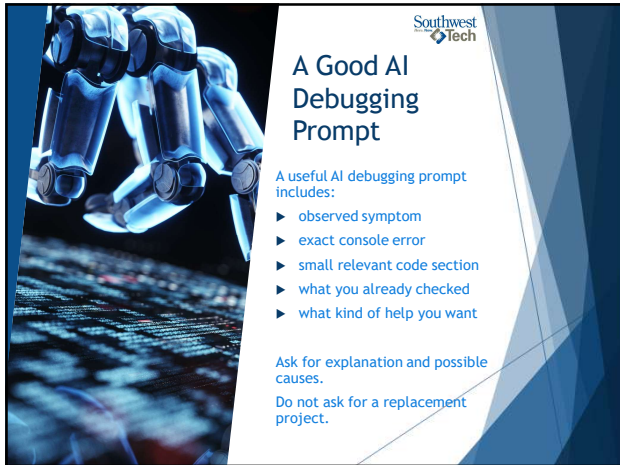
Fix 2
Make one targeted change. → Retest (Run the same action or test again). → Evidence (Record what happened and what changed).

Fix 3
Make one targeted change. → Retest (Run the same action or test again). → Evidence (Record what happened and what changed).

Retesting turns edits into evidence.
Change one cause. Retest the same action. Record what you see. Use results to choose the next issue.

Retest After Each Fix

21



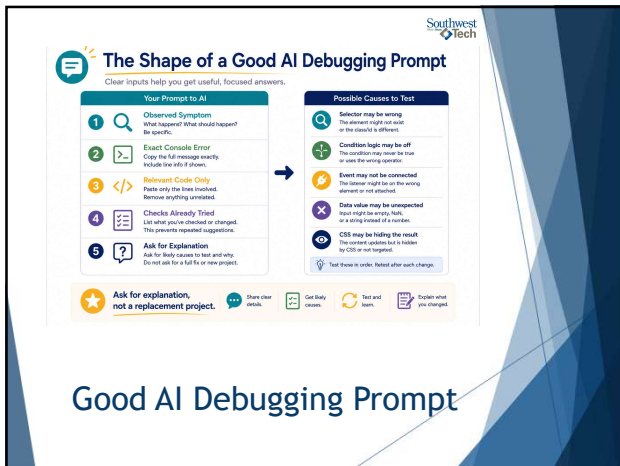
A Good AI Debugging Prompt

A useful AI debugging prompt includes:

- ▶ observed symptom
- ▶ exact console error
- ▶ small relevant code section
- ▶ what you already checked
- ▶ what kind of help you want

Ask for explanation and possible causes.
Do not ask for a replacement project.

22



The Shape of a Good AI Debugging Prompt

Clear inputs help you get useful, focused answers.

Your Prompt to AI

- Observed Symptom**
What happened? What should happen? Be specific.
- Exact Console Error**
Copy the full message exactly. Include line info if shown.
- Relevant Code Only**
Paste only the lines involved. Remove anything unrelated.
- Checks Already Tried**
List what you've checked or changed. This prevents repeated suggestions.
- Ask for Explanation**
Ask for likely causes to test and why. Do not ask for a full fix or new project.

Possible Causes to Test

- Selector may be wrong**
The element might not exist or the class/id is different.
- Condition logic may be off**
The condition may have the logic or uses the wrong operator.
- Event may not be connected**
The listener might be on the wrong element or not attached.
- Data value may be unexpected**
Input might be empty, null, or a string instead of a number.
- CSS may be hiding the result**
The current styles but it is hidden by CSS or not targeted.

Ask for explanation, not a replacement project.

Other clear checks: Get body classes, Test and learn, Explain what you changed

23



Debugging Report Pattern

Debugging report pattern:

Issue:

- ▶ what was wrong?

Evidence:

- ▶ how did you identify it?

Fix:

- ▶ what did you change?

Verification:

- ▶ how do you know it works now?

24



Debugging Report

Name: _____ Date: _____ Assignment/Task: _____

- Issue:**
What was wrong?
(Write 1-2 sentences.)
- Evidence:**
How did you identify it?
(Console output, console log output, browser behavior, etc./See checkbox)
- Fix:**
What did you change?
(Name the file, line number, and change.)
- Verification:**
How do you know it works now?
(Print result, expected behavior, no console error, etc.)

★ Fixed code alone is not the whole assignment. Your reasoning, evidence, and verification matter.

Debugging Report Pattern

25



Lab Refinement

Verified fixes
Your goal:

- ▶ fix at least 2 confirmed issues completely
- ▶ use console output or inspection as evidence
- ▶ verify the behavior after each fix
- ▶ document issue, evidence, fix, and verification

26



Evidence & Reflection

Evidence:

- ▶ updated HTML, CSS, and JS
- ▶ site runs with fixes applied
- ▶ at least two issues fixed correctly
- ▶ debugging report included

Reflection

- ▶ Explain how your problem-solving changed

27

Southwest
Tech

How To Read Next Week's Material



Required:

- ▶ read functions as a way to organize behavior
- ▶ notice scope, dot notation, and document as a built-in object

Reference:

- ▶ objects and built-in methods are there when you need them
- ▶ do not memorize every method

Before next time:

- ▶ Bring one piece of interaction code that could be made clearer.

28

Southwest
Tech

From Debugging to Cleaner Behavior

A clear path from problem to better code.



1 Bug Found
Something is not working as expected. Notice the program. Capture the evidence.

2 Cause Understood
You know why it happened. Connect the evidence to the root cause.

3 Fix Verified
The fix works and behaves correctly. Retest. Confirm the expected result.

4 Code Made Cleaner
Improve readability and structure. Better names, simpler logic, clear intent.

Working code becomes cleaner code next.

Find the bug. Understand the cause. Verify the fix. Make the code cleaner.

Cleaner Code Bridge

29

Southwest
Tech

Closing Success Check



This week:

- ▶ problems became evidence
- ▶ console messages became clues
- ▶ fixes were verified
- ▶ reports explained the process

Next:

- ▶ working code becomes cleaner code.

30



31
